

MENG INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Data Poisoning Attacks against Adaptive (Continual) Learning

Author:
Dillan Scott

Supervisor:
Prof. Emil Lupu

Second Marker:
Prof. Sergio Maffei

June 2, 2026

Abstract

Todo

Acknowledgements

Todo

Contents

1	Introduction	7
1.1	Context and Motivations	7
1.2	Scope	8
1.3	Objectives	8
1.4	Research Questions	8
1.5	Thesis Structure	8
2	Background	10
2.1	Concept Drift	10
2.1.1	Taxonomy of Drift	10
2.2	Concept Drift Detection	11
2.2.1	General Architecture of Drift Detection Systems	11
2.2.2	Drift Detection Algorithms	12
2.2.3	Limitations and Weaknesses	13
2.3	Adversarial Concept Drift	14
2.3.1	Adversarial Objectives	14
2.3.2	Instance-Based Poisoning	15
2.3.3	Concept-Based Poisoning	15
2.4	Adversarially Robust Solutions	15
2.4.1	Robust Generative Modelling (RRBM-DD)	15
2.4.2	Contrastive Learning and Hybrid Distance Metrics	16
3	Target System and Threat Model	17
3.1	Target System Architecture	17
3.1.1	The Base Learner	17
3.1.2	The Drift Detector	17
3.1.3	System Integration and Adaptation Cycle	18
3.2	Threat Model	18
3.2.1	Attacker Objectives	18
3.2.2	Attacker Knowledge	18
3.2.3	Attacker Capabilities	19
4	Attack Taxonomy and Objectives	20
4.1	Axis 1: Attacks on the Base Learner (M)	20
4.2	Axis 2: Attacks on the Drift Detector (DD)	20
4.3	The Maintenance Principle	21
4.3.1	Meaningful Single-Component Attacks	21
4.4	Compound Attacks: The Matrix of Combinations	21
4.4.1	The Primary Threats	21
4.4.2	Pointless or Impractical Combinations	22
5	Implementation	23
6	Evaluation	24
7	Conclusion	25

8	Ethical Issues	26
8.1	The Dual-Use Dilemma and Offensive Security	26
8.2	Legal and Professional Compliance	26
8.3	Data Governance and Privacy	26
A	First Appendix	27

List of Figures

2.1	Examples of concept drift types based on temporal patterns	11
3.1	The general architecture of the target system	18

List of Tables

Todo list

	Todo	1
	Todo	1
	Rewrite introduction chapter to reflect new goals	7
	Switch order and include some details about the NCM approach	7
	Finish this paragraph	7
	More here?	8
	Think about more relevant inclusions in this chapter	10
	Maybe remove if no longer relevant	15
	Rewrite section to be way more in-depth	16
	Detail the exact adaptation mechanism here: e.g., Does it buffer new data to train a replacement tree in the background? Does it immediately discard the old tree?	18
	Figure can be improved	18
	Maybe mention something about the maintenance of attacks	20
	Write a subsection here (include attacks that require no DD manipulation - ride real drift signal)	21
	Break into more subcategories	21
	Rewrite ethics chapter	26

Chapter 1

Introduction

1.1 Context and Motivations

It is becoming increasingly necessary for machine learning systems to operate in dynamic environments where data arrives as a stream with no guarantees about its underlying distribution. Examples include fraud detection, network intrusion detection, and content moderation, where new observations are generated sequentially, and historical data may eventually become outdated.

Learning under these conditions is referred to as adaptive or continual learning. In contrast to traditional batch learning, where models are static post-deployment, adaptive models are repeatedly updated. The motivation for these online updates is not only to handle the volume or velocity of streaming data, but also to account for concept drift - changes in the joint distribution of inputs and outputs over time. The performance of a static model, which is accurate on current data, may degrade as the environment changes. Consequently, mechanisms known as drift detectors are used to monitor incoming data streams and signal the need for model updates when necessary.

While drift detection has been extensively studied in non-adversarial settings [1, 2, 3], critical security concerns arise when operating in environments where strategic or malicious actors may influence data streams. In domains such as fraud detection and cybersecurity, adversaries may deliberately manipulate data to evade detection, degrade model performance, or force harmful model adaptations. Unlike poisoning attacks against fixed models, adversarial attacks against adaptive systems can aim to exploit the update mechanism - using it as a vector to inject malicious concepts, suppress valid adaptations, or corrupt the detector's learned representation.

In response to this threat, recent research has proposed adversarially robust drift detectors, including methods that combine a contrastive loss objective with Nearest Class Mean (NCM) classifiers [4], and those using reconstruction error with energy-based models such as Restricted Boltzmann Machines [5]. These approaches aim to distinguish organic distributional change from adversarial poisoning or noise, and they have demonstrated robustness against certain poisoning strategies.

However, both works share a fundamental limitation when viewed from the perspective of a fully coupled adaptive learning pipeline. Kuppa and Le-Khac [4] evaluate their detector in isolation: their experiments measure the detector's ability to correctly classify incoming samples as drifted or not, reporting precision, recall and F1 against held-out drifted classes. No downstream base learner is used, meaning their robustness claims speak only to the detector's own classification performance and say nothing about whether those properties extend to protecting a separate model that depends on the detector to govern its adaptation. Korycki and Krawczyk [5] do include a base learner, but they choose a model that updates continuously, instance by instance, regardless of drift signals; meaning the detector is advisory rather than controlling.

This discrepancy is significant... Understanding these limitations...

Rewrite introduction chapter to reflect new goals

Switch order and include some details about the NCM approach

Finish this paragraph

1.2 Scope

This thesis examines adversarial attacks against an adaptive learning pipeline consisting of a static MLP classifier coupled with the contrastive auto-encoder and Nearest Class Mean detector [4]. The work is centred on understanding whether this state-of-the-art robust detector is sufficient to protect a trigger-dependent base learner when the two components are deployed together as a coupled system.

This work takes an adversarial perspective, proposing attacks against the coupled system, covering those on each element both individually and simultaneously. A representative subset of these attacks is then implemented and evaluated, selected to span the primary ways in which an adversary could exploit the system’s coupling.

The aim is not to refute the robustness claims of Kuppa and Le-Khac [4] but to test whether those claims extend to the protection of a downstream base learner in a realistic adaptive learning architecture. By doing so, the thesis seeks to clarify the practical security guarantees offered by the detector and identify the conditions under which the coupled system remains vulnerable.

1.3 Objectives

The primary objective of this research is to determine whether the robustness of the contrastive auto-encoder and NCM detector is sufficient to protect a static, trigger-dependent base learner from corruption when the two components are deployed as a coupled adaptive learning system.

First, it formalises a taxonomy of attacks against the coupled system and identifies the structural properties of trigger-dependent architectures that create attack surfaces absent from prior evaluations. Second, it implements and experimentally analyses whether each attack can successfully harm the base learner’s predictive capabilities.

More
here?

1.4 Research Questions

To meet the objective described in the previous section, this thesis addresses the following questions:

1. How can the capabilities and knowledge of an adversary targeting a coupled adaptive learning system be formally defined, and what attack surfaces are introduced by trigger-dependent base learner architectures that are absent when the base learner updates continuously?
2. Can an adversary successfully corrupt the predictive accuracy of a static base learner by exploiting the retraining mechanism of a system equipped with the contrastive auto-encoder and NCM detector, despite its robustness properties?
3. Can an adversary corrupt the detector’s learned representation sufficiently to manipulate the adaptation of the base learner — either by forcing false retraining events or by suppressing valid ones?
4. What ongoing poisoning budget, in instance count and perturbation magnitude, is required to maintain these adversarial effects against a detector that continuously updates its representation of the data distribution?
5. What is the trade-off between the detector’s robustness to adversarial poisoning and its sensitivity to genuine concept drift, and how does this manifest in a trigger-dependent coupled architecture?

1.5 Thesis Structure

The remainder of this thesis is organised as follows:

- [Chapter 2](#) reviews the background literature on continual learning and adversarial attacks.
- [Chapter 3](#) formalises the target system and establishes the threat model, defining the adversary’s objectives, knowledge, and capabilities.

- [Chapter 4](#) presents a taxonomy of attacks against the coupled system and introduces the idea of maintenance cost.

Chapter 2

Background

Think about more relevant inclusions in this chapter

This chapter establishes the theoretical foundations and reviews the relevant literature in order to contextualise the latter contents of this thesis. It begins by formally defining concept drift and outlining its taxonomy based on probabilistic decomposition and temporal characteristics. Subsequently, the general architecture of drift detection systems is analysed, alongside a review of some existing concept drift detectors and their inherent limitations. The chapter then moves on to consider a security perspective, introducing the notion of adversarial concept drift, where distributional shifts are manipulated by potentially malicious actors. Finally, the chapter concludes by discussing the already existing robust solutions, which have been proposed to mitigate these challenges.

2.1 Concept Drift

Concept drift generally refers to changes in the statistical relationship between input features and target labels, characterised as a change in the true joint distribution $p(X, y)$ [1]. Formally, concept drift between time points t_0 and t_1 is said to occur when

$$p_{t_0}(X, y) \neq p_{t_1}(X, y), \quad (2.1)$$

where p_{t_0} and p_{t_1} denote the joint distributions of the input variables X and target variable y at times t_0 and t_1 , respectively [2].

2.1.1 Taxonomy of Drift

Probabilistic Nature

The joint probability can be given by the decomposition

$$p(X, y) = p(y | X)p(X) \quad (2.2)$$

where $p(y | X)$ is the conditional distribution of the target given the input features and $p(X)$ is the probability distribution of the input features. Based on this, the literature [1, 2, 5] distinguishes between two forms of concept drift:

1. *Real Drift*: When $p(y | X)$ changes without a change in $p(X)$ (Conditional Change) or with it (Dual Change). This is the more critical form of drift, as it directly affects decision rules or classification boundaries, making model adaptation necessary.
2. *Virtual Drift (Feature Change)*: When $p(X)$ changes without affecting $p(y | X)$. While this may appear less important than real drift because the decision function is left unchanged, it cannot be ignored. In particular, changes in $p(X)$ may trigger adaptation or forgetting mechanisms, potentially leading to unnecessary or even harmful model updates.

Temporal Characteristics

In addition to the above types, based on probabilities, concept drift can be categorised based on how the change occurs over time. Namely, four common patterns are shown in Fig. 2.1 [6]

1. *Sudden/Abrupt drift* is when the instance distribution or concept changes within a very short time frame. The previous concept immediately becomes invalid, and the learner must adapt quickly to maintain performance [5].
2. *Gradual drift* is where instances from the old and new concepts coexist for a period, with old appearing with decreasing frequency [5].
3. *Incremental drift* is when there is a smooth and continuous progression from one concept to another through a series of intermediate concepts [2, 5].
4. *Reoccurring concepts* refers to when previously seen concepts reappear after some time, such as in seasonal or cyclical patterns [2]. They may reappear both suddenly or incrementally [7].

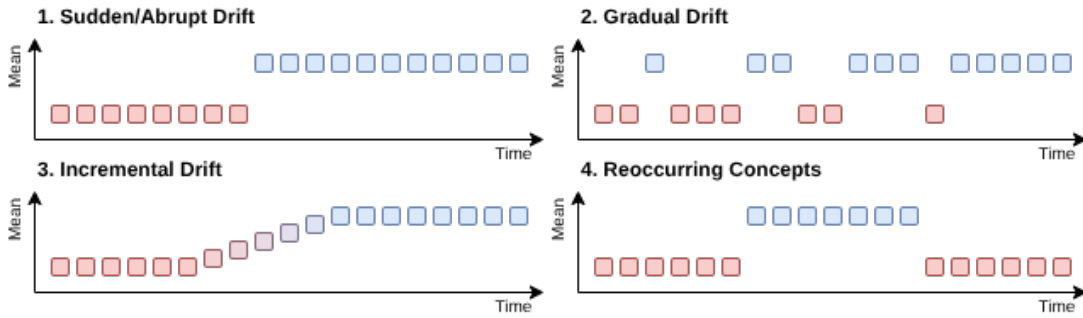


Figure 2.1: Examples of concept drift types based on temporal patterns

2.2 Concept Drift Detection

In data stream learning, concept drift is typically addressed by augmenting the predictive model (the base learner) with a drift detection mechanism. In general, drift detectors attempt to identify changes in the data distribution by analysing the base learner’s behaviour [3]. Rather than continuously retraining the learner, drift detectors act as a supervisory mechanism, triggering an adaptation process, such as model retraining, parameter update, or selective forgetting of historical data.

2.2.1 General Architecture of Drift Detection Systems

Although they are algorithmically diverse, most concept drift detection systems follow a common architectural pattern. This architecture can be decomposed into four conceptual components:

1. *Observation Source*: The detector monitors a stream of signals derived either from the data itself or from the performance of a base learner. Common sources include error rates, loss values, feature statistics, or latent representations.
2. *Summary Mechanism*: Observations are aggregated over time using sliding windows, cumulative statistics, or incremental estimators. This step compresses the raw stream into a tractable representation that reflects the system behaviour of a given period.
3. *Change Test*: A statistical test, threshold comparison, or learned decision rule evaluates whether the observed summary deviates significantly from a reference/historical summary. This comparison may rely on bounds derived from concentration inequalities, hypothesis tests, or reconstruction error metrics.
4. *Decision and Response*: Upon detecting significant deviation, the detector emits a signal (sometimes distinguishing between a warning and confirmed drift) which triggers downstream actions such as retraining, window resetting, or model replacement.

This modular structure highlights an important property of drift detectors: they act as *control mechanisms* governing when and how adaptive learning occurs. As such, their behaviour has long-term consequences for system performance, stability, and robustness.

2.2.2 Drift Detection Algorithms

While the general architecture remains consistent, specific implementations differ in their observation sources and statistical tests. The literature typically categorises these methods into error rate-based and data distribution-based approaches [6].

Error Rate-Based Detectors

These methods monitor the online performance of the base learner (e.g. classification accuracy). They function based on the assumption that a statistically significant increase or decrease in model performance implies a change in the underlying data distribution [6]. Drift detectors in this category include:

- *Drift Detection Method (DDM)* [8]: This method models the number of errors as a random variable from a Binomial distribution. It tracks the online error rate p_t and its standard deviation $s_t = \sqrt{p_t(1-p_t)/t}$ at time t . The algorithm maintains the minimum values observed so far, p_{min} and s_{min} , updating them whenever $p_t + s_t < p_{min} + s_{min}$. A warning level is triggered if:

$$p_t + s_t \geq p_{min} + 2s_{min}. \quad (2.3)$$

at which point incoming examples are buffered while using the old learner for predictions. If the error rate continues to rise and drift is confirmed via the condition:

$$p_t + s_t \geq p_{min} + 3s_{min} \quad (2.4)$$

a new learner is induced using the buffered examples, and the old learner is discarded.

- *ADaptive WINdowing (ADWIN)* [9]: This method uses a sliding window W that dynamically adjusts its size, based on the rate of change in the data stream. It operates by examining all possible cuts of the current window into two sub-windows, W_0 and W_1 , representing historical and newer data, respectively. Drift is detected if the absolute difference between the averages of these sub-windows is statistically significant:

$$|\hat{\mu}_{W_0} - \hat{\mu}_{W_1}| \geq \epsilon_{cut} \quad (2.5)$$

where ϵ_{cut} is an adaptive threshold (determined using Hoeffding bounds and a confidence interval δ). When this threshold is reached, the historical data (W_0) is discarded as the two sub-windows are seen to have come from different data distributions. The author also proved that the false positive and false negative rates are bounded by δ .

Data Distribution-Based Detectors

In contrast, these methods monitor the statistical properties of the input features ($P(X)$), independent of the base learner's behaviour and error rate. They rely on the assumption that drift can be signalled by a statistically significant deviation in the distribution of new data when compared to a historical reference window [6]. Because they analyse the input data itself, these methods can detect virtual drift that does not immediately alter the decision boundary and can also function when ground truth labels are unavailable. Examples of data distribution-based detectors include:

- *Statistical Change Detection for multidimensional data (SCD)* [10]: This detector addresses the challenge of multidimensional distribution change by employing a non-parametric statistical test known as the Density Test. The baseline/historical window S is partitioned into two halves: S_1 for modelling and S_2 for testing. It utilises a Kernel Density Estimator (KDE) on S_1 , to get the density function $\mathcal{K}_{S_1}$. The test statistic δ is then defined using the difference in log-probability density of $\mathcal{K}_{S_1}$ at S' and at S_2 . Specifically:

$$\delta = LLH(\mathcal{K}_{S_1}, S') - \frac{|S'|}{|S_2|} \times LLH(\mathcal{K}_{S_1}, S_2) \quad (2.6)$$

where LLH is the log-likelihood. Drift is confirmed if δ falls below a critical value C_{max} (algorithmically derived from a user-defined significance level p).

- *Principal Component Analysis-Based Change Detection (PCA-CD)* [11]: This method works on multidimensional data streams by projecting the data onto lower-dimensional spaces using PCA. It extracts the top k principal components that explain a substantial portion of the variance from the historical window and then monitors new data projected onto these components as separate univariate streams. Similarly to SCD, the density of these projections is estimated using KDE, for example. A divergence metric, such as the intersection area under the curves of the reference ($\hat{f}$) and test ($\hat{g}$) distributions:

$$D_A(\hat{g} \parallel \hat{f}) = 1 - \int_x \min(\hat{f}(x), \hat{g}(x)) dx \quad (2.7)$$

is calculated for each of the k components. Then the final change score is taken as the maximum of these. To detect drift, the method applies the Page-Hinkley (PH) test with a dynamic threshold to the aggregate score, triggering an alarm when the cumulative deviation from the historical mean exceeds a given limit.

2.2.3 Limitations and Weaknesses

While the drift detectors described above do provide robust mechanisms for adaptation, they are subject to inherent limitations and trade-offs between sensitivity (minimising false negatives) and specificity (minimising false positives). Rather than arising from flawed algorithms, many of these weaknesses are due to the statistical assumptions made by classical drift detection. Most notably, finite-sample estimation, approximate stationarity within windows, and the use of fixed confidence bounds to identify change.

Importantly, these limitations can be mapped onto the general architecture of drift detectors introduced earlier in [Section 2.2.1](#). Choices made at the level of the *observation source* (e.g. error rates or feature statistics), the *summary mechanism* (e.g. windowing or cumulative statistics), and the *change test* (e.g. hypothesis tests or thresholds) impose constraints on what types of drift can be reliably identified and how quickly.

Sensitivity to Gradual Drift

A limitation of some error-based detectors, such as DDM, is the reduced ability to distinguish between stationary noise and gradual concept drift. In the presence of abrupt drift, the error rate typically exhibits a sharp increase, triggering the detector’s thresholds (defined for DDM in [Eq. \(2.3\)](#) and [Eq. \(2.4\)](#)). However, during a gradual drift, the error rate may increase too slowly to violate the statistical bounds.

Under such conditions, the detector may update its internal reference statistics (p_{min} and s_{min} in the case of DDM), effectively incorporating the drifted distribution into its baseline instead of identifying it. While adaptive windowing approaches, such as ADWIN, mitigate this by dynamically adjusting the window size, they are still bounded by the resolution of their sliding window. Drift, which remains below the statistical bound (defined for ADWIN as ϵ_{cut} in [Eq. \(2.5\)](#)), will remain undetected until enough samples accumulate to reach statistical significance.

Fundamentally, this limitation highlights the difficulty in distinguishing between slow drift and stationary noise under bounded confidence levels, without making additional assumptions about the drift process. Any attempt to increase sensitivity by lowering detection thresholds inevitably raises the false positive rate, highlighting a trade-off key to all hypothesis test-based or threshold-based drift detectors.

Verification Latency and Label Availability

Error-based detectors suffer from a critical dependency on ground truth availability. They assume that labels are available immediately or with minimal delay after the prediction is made, to compute prediction errors and monitor performance degradation. In many real-world streaming applications, such as credit fraud detection or loan forecasting, the true label may arrive with significant delay (verification latency) or may never arrive at all.

During periods of high verification latency, the detector effectively cannot observe its primary signal, leaving the system blind to changes in performance. If drift occurs during such a period, the system will continue to trust an outdated model, accumulating loss until detection becomes possible.

Virtual Drift and False Positives

Distribution-based detectors address the problem with label dependency by monitoring the input feature distribution $P(X)$ rather than model error. While this enables detection in settings with delayed or missing labels, it introduces a different weakness related to virtual drift. A statistically significant change in $P(X)$ does not always imply a change in the conditional distribution $P(y | X)$. It therefore does not guarantee that the predictive decision boundary has changed.

Consequently, distribution-based detectors may generate false positives, triggering model retraining or data forgetting even when performance remains stable (there is no *real* drift). Such unnecessary adaptation can be computationally expensive and may temporarily degrade performance.

Dimensionality and Feature Relevance

In high-dimensional settings, distribution-based detectors rely on density estimation or projection techniques that are sensitive to the curse of dimensionality. As the number of dimensions increases, data sparsity reduces the reliability of density estimates, while dimensionality reduction risks discarding directions along which drift is relevant. Moreover, distribution-based detectors typically treat all features with equal importance, meaning they are susceptible to changes in weakly relevant features that may not materially affect model performance.

2.3 Adversarial Concept Drift

Classical concept drift assumes that changes in the data distribution arise from natural, exogenous factors such as evolving user behaviour, seasonality, or changes in the underlying environment. In contrast, adversarial drift assumes that there is a purposeful and potentially malicious actor attempting to manipulate our adaptive learning system [5]. The potential goals they may have will be discussed in the next section.

In such settings, the drift process is no longer passive. Instead, the data stream itself becomes an attack surface, and the drift detector becomes a target. Adversarial drift extends traditional data poisoning attacks by exploiting temporal dynamics and specific detector behaviours. Rather than corrupting a static training set, the adversary seeks to control *when* adaptation is triggered, *what data* is included during retraining, and *which information* is forgotten. This introduces a new class of threats that cannot be fully explained by classical adversarial learning models alone [12].

2.3.1 Adversarial Objectives

An adversary interacting with an adaptive learning system may pursue several objectives, which are not mutually exclusive.

A primary objective could be *performance degradation*, where the adversary seeks to reduce the predictive accuracy or increase the loss of a system [13]. Unlike poisoning attacks, adversarial drift may be gradual, ensuring that degradation accumulates slowly while remaining below detection thresholds.

This could be seen as a potential secondary objective: *evasion of detection*. Here, the adversary aims to avoid the activation of the drift detector while pursuing their primary objective. This could involve crafting inputs that move the decision boundary while preserving the statistics monitored, hence exploiting the blind spots of classical detectors.

Alternatively, an adversary may attempt *forced adaptation*, deliberately triggering false drift alarms to cause unnecessary retraining or forgetting. Such behaviour can destabilise the learning process, increase computational costs, or induce catastrophic forgetting of valid historical concepts [5]. Conversely, an adversary could seek to *block adaptation*, injecting contradictory data points during a period of valid concept drift, in an attempt to nullify the adaptation process [5].

Finally, adversaries may seek *control over the learned concept*, gradually forcing the system to learn a decision boundary favourable to the attacker's goals. In this case, the detector is actively exploited as a mechanism for injecting the poisoned instances during adaptation cycles.

2.3.2 Instance-Based Poisoning

Instance-based poisoning attacks are those in which an adversary injects or modifies individual data points in the stream to influence the base learner or drift detector. These attacks are powerful because the poisoned instances can be distributed over time, allowing adversaries to exploit both the learner's update behaviours and the detector's temporal assumptions. For example, poisoned samples may be injected intermittently or gradually, ensuring that individual instances appear benign while the cumulative effect leads to performance degradation or detector manipulation.

From a drift detection perspective, instance-based poisoning can be used to:

- *Bias summary statistics*: By subtly increasing error rates or similar statistics within certain bounds, the adversary can influence the detector's inner mechanisms.
- *Trigger false alarms*: Bursts of or extreme poisoned instances may surpass thresholds, triggering unnecessary adaptation or window resets.
- *Suppress true drift detection*: Conversely, carefully engineered samples can stabilise monitored statistics during periods of genuine drift, delaying or preventing detection [5].

Crucially, even small, strategically placed perturbations can be sufficient to accumulate and have the desired effect, particularly against detectors based on hypothesis testing or confidence bounds. This makes instance-based poisoning difficult to distinguish from natural noise or gradual drift, especially in high-variance data streams.

2.3.3 Concept-Based Poisoning

Concept-based attacks function at a higher level of abstraction than instance-based attacks. Rather than focusing on individual samples, the adversary aims to directly mimic a shift in the data-generating process to falsify genuine concept drift. The adversary engineers the drift trajectory to achieve a desired outcome, such as the misclassification of specific regions of the input space or the biased treatment of certain inputs.

By anticipating how the detector responds to changes in monitored statistics, the adversary can:

- *False adaptation*: Triggering adaptation where the system interprets the adversarial concept as a genuine distributional change [5].
- *Exploit forgetting mechanisms*: Causing the system to discard historically valid or important data.
- *Suppressed adaptation*: Balancing genuine concept drift with a conflicting distribution, preventing the detector from reaching its change threshold or negatively impacting the adaptation process.

2.4 Adversarially Robust Solutions

To address the vulnerabilities of traditional detectors, recent research proposes methods explicitly designed to withstand adversarial manipulation. These solutions leverage techniques like contrastive embedding and generative modelling to differentiate between natural and adversarial distributional shifts.

2.4.1 Robust Generative Modelling (RRBM-DD)

Korycki and Krawczyk [5] introduce the Robust Restricted Boltzmann Machine for Drift Detection (RRBM-DD), a generative neural network that detects drift by monitoring the reconstruction error. Unlike classical models, the RRBM-DD incorporates specific mechanisms in an attempt to filter out malicious data during updates.

Maybe
remove if
no longer
relevant

To counter instance-based poisoning, it uses a robust gradient descent method with M-estimators that softly truncate the influence of extreme loss values, preventing outliers from skewing the weight updates. For concept-based poisoning, the model employs a robust energy function with a gating mechanism and a Gaussian noise model. This allows the network to dynamically turn off neurons affected by adversarial clusters, ensuring the detector remains sensitive to genuine drift while ignoring malicious artefacts.

Rewrite
section
to be
way
more in-
depth

2.4.2 Contrastive Learning and Hybrid Distance Metrics

Kuppa and Le-Khac [4] propose a detector that maps high-dimensional inputs to a low-dimensional latent space using a contrastive loss objective. To create tighter class clusters, which are naturally more robust to adversarial perturbations, this objective places similar samples closer together while spreading dissimilar ones apart.

Drift detection is then performed on this latent space using a Nearest Class Mean (NCM) classifier equipped with a hybrid distance metric. This metric combines both Euclidean distance (instance-level fidelity) and Riemannian divergence (group-level geometric fidelity). The approach relies on the fact that adversarial subspaces typically lie on low-probability manifolds distinct from the true data distribution. Additionally, the system supports a recursive update mechanism which automatically discovers and defines new classes from accumulated drift vectors.

Chapter 3

Target System and Threat Model

This chapter formalises the adaptive learning pipeline targeted in this research and defines the capabilities and knowledge of an adversary attempting to compromise it. First, the architecture of the target system is detailed, consisting of a base learner coupled and an adversarially robust drift detector. Subsequently, a complete threat model is established, specifying the attacker’s objectives, assumed knowledge, and operational capabilities.

3.1 Target System Architecture

The system under evaluation addresses concept drift in the typical way: by augmenting the base learner with a drift detector, as mentioned in [Section 2.2](#). However, this approach differs in that the supervisory module is dynamic. While traditional detectors rely on fixed statistical thresholds or heuristics, this architecture utilises a trainable drift detector. It learns a compressed latent representation of the known data distribution and measures deviation using class prototypes in that space to identify distributional shifts. This approach is intended to provide robustness to adversarial manipulation, a property which is scrutinised in the following chapters of this thesis.

The two components of the system, and the manner in which they are integrated, is described below.

3.1.1 The Base Learner

The base learner $\mathcal{M}$ is a binary multi-layer perceptron trained to classify the network flows as either BENIGN (class 0) or ATTACK (class 1). It consists of three fully-connected hidden layers of width 256, 128, 64 respectively, followed by a two-class output layer, giving:

$$\mathcal{M} : \mathbb{R}^d \rightarrow \mathbb{R}^2$$

3.1.2 The Drift Detector

The drift detector $\mathcal{D}$ is the Contrastive Nearest Class Mean Detector (ContrastiveNCM) [4]. Its architecture and training procedure are described in full in [Section 2.4.2](#); here we only discuss the configuration choices specific to this experimental setup.

The encoder is trained with a latent dimension of $r = 32$, a hidden layer of width 64, and a contrastive temperature of $\tau = 0.1$, matching the hyperparameters reported in the original paper. The concept-discovery threshold is set to $T = 3.5$, again following the paper[4].

Drift threshold calibration

The distance threshold τ_{drift} , which determines whether an individual sample is flagged as drifted, is not fixed a priori. After initial training on the known-class data, it is set to the 95th percentile of NCM distances computed on a held-out calibration split containing only known-class instances. This yields a nominal false-positive rate of approximately 5% on in-distribution traffic

while correctly flagging novel-class samples, which are far from all known prototypes in the latent space. Crucially, the calibration requires no labels from novel classes, which is consistent with the deployment constraint that novel attacks have not yet been observed at training time.

3.1.3 System Integration and Adaptation Cycle

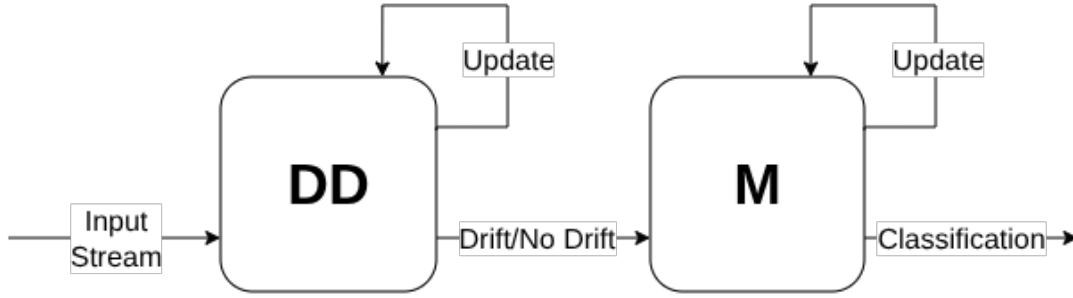


Figure 3.1: The general architecture of the target system

3.2 Threat Model

To systematically evaluate the security of the target system, it is necessary to define the parameters under which an adversary operates. The threat model establishes the attacker’s objectives, their level of knowledge about the system, and their practical capabilities within the environment.

3.2.1 Attacker Objectives

The primary objective of an adversary is to compromise the predictive accuracy or operational efficiency of the system as a whole. In this context, this manifests in two distinct ways:

- *Forcing False Adaptation:* The attacker aims to deceive the RRB-DD into signalling a drift when the underlying distribution is stable, causing the Decision Tree to unnecessarily adapt to potentially malicious data, degrading its accuracy or consuming computational resources.
- *Impairing Adaptation:* The attacker injects data that masks genuine concept drift in some way, preventing the RRB-DD from detecting valid shifts and forcing the Decision Tree to rely on an outdated model.

3.2.2 Attacker Knowledge

The viability of an attack heavily depends on the adversary’s understanding of the target system. The attacker’s knowledge can be framed under 3 different assumptions:

- *White-Box:* The adversary possesses complete knowledge of the system. This includes but is not limited to the RRB-DD’s exact architecture, its current weights, the size of the mini-batches, and the specific statistical thresholds used in the Granger causality test.
- *Grey-Box:* The adversary knows the general system architecture (e.g. that an RRB-DD and a Decision Tree are being used) and the features of the data stream, but lacks access to the internal weights, sliding window states, or exact hyperparameters in real-time.
- *Black-Box:* The adversary is unaware of the specific detection algorithms or base learner implemented. They can only observe the output predictions and the raw incoming data stream, meaning they must infer further details like the occurrence of a drift signal, based on changes in the system’s external behaviour.

3.2.3 Attacker Capabilities

In the context of data stream mining, the adversary is assumed to have write access to the data ingestion pipeline, allowing them to manipulate the stream before it enters the predictive system. The adversary's capabilities are defined by the following constraints:

- *Feature Manipulation*: The ability to perturb the feature vector X of incoming data points.
- *Label Flipping*: The ability to alter the ground-truth label y provided to the system.
- *Injection Budget*: An adversary cannot replace 100% of the stream, as this would trivially overwrite the system and would likely be flagged by upstream anomaly detection.

Chapter 4

Attack Taxonomy and Objectives

This chapter systematically explores the attack vectors that can be deployed against the target continual learning pipeline. Because the architecture couples a base learner (M) with a trainable drift detector (DD), its security must be evaluated across multiple dimensions. We introduce a 3×3 attack matrix, categorising threats along two axes: attacks on the base learner and attacks on the drift detector.

Maybe mention something about the maintenance of attacks

4.1 Axis 1: Attacks on the Base Learner (M)

Attacks on the base learner (the Decision Tree) aim to degrade its predictive performance, either during inference or by exploiting retraining.

- *Evade M* : The adversary crafts perturbations to input features to force a misclassification, bypassing the current decision boundary without altering the model itself. Occurs strictly during the inference phase.
- *Poison M* : Exploits system adaptation by ensuring corrupted data is included in the retraining buffer when a drift is signalled, forcing M to learn a false concept and degrading its accuracy.
- *Backdoor M* : A targeted form of poisoning where the adversary injects a feature "trigger" paired with a target label into the retraining buffer. After the update, M behaves normally on clean data but behaves as the attacker specified for inputs containing the hidden trigger.

4.2 Axis 2: Attacks on the Drift Detector (DD)

Attacks on the RRB- DD aim to hijack the system's supervisory mechanism, dictating when and if the system adapts.

- *Evade DD* : Causes misclassification without needing to alter the weights of DD . This manifests as either a False Negative (masking real drift) or a False Positive (triggering unnecessary retraining).

As a stand-alone attack, a stream of False Negatives can be used to stop M from learning, and a stream of False Positives can be used as a form of Denial of Service by causing repeated retraining.

- *Poison DD* : Exploits the fact that RRB- DD 's trains itself continually. The adversary injects adversarial concepts that bypass the network's robust gradient and noise filters, corrupting its understanding of the distribution.

This can be used alone to increase or decrease the detector's tolerance to changes, causing blindness or hypersensitivity to real drift, respectively.

- *Backdoor DD*: Plants a hidden trigger within the RRBm-DD’s latent representation by exploiting its online updating mechanism. This establishes a targeted vulnerability without corrupting the detector’s baseline performance.

This alone can grant an attacker precise, on-demand control over the system, allowing them to dictate exactly when a drift is detected and when a retraining cycle is initiated or suppressed.

4.3 The Maintenance Principle

Because the target architecture relies on a trigger-based adaptation mechanism, any malicious payload injected into the system is inherently temporary. Continual learning systems naturally evolve; therefore, even if an attacker successfully poisons the base learner (M), subsequent natural concept drift will trigger a new retraining cycle and overwrite the compromised model.

Therefore, any poisoning or backdoor placed in either DD or M must be actively maintained. When attacking DD , the adversary must continuously inject the adversarial distribution into the stream to prevent the neural network from healing and relearning the true data distribution. When attacking M , the adversary must protect the compromised model by suppressing future drift signals to prevent M from retraining. This suppression can be achieved by actively evading DD (forcing false negatives), poisoning DD to raise its tolerance, or utilising a backdoor in DD to dictate its signalling behaviour.

4.3.1 Meaningful Single-Component Attacks

...

4.4 Compound Attacks: The Matrix of Combinations

By combining the three strategies for M (Evade, Poison, Backdoor) with the three strategies for DD , we yield nine potential compound attacks. The following subsections categorise these combinations based on their practical viability and threat level.

4.4.1 The Primary Threats

These combinations are both viable and present a threat.

- *Evade DD + Poison M*: The attacker injects data that causes DD to falsely signal a drift. M is subsequently retrained on a batch of poisoned instances injected concurrently by the attacker. The attacker must then maintain this poisoned state using the methods discussed in [Section 4.3](#).
- *Evade DD + Backdoor M*: Mechanically similar to the attack above, but rather than degrading general accuracy, the attacker implants a specific feature trigger into M during the forced retraining cycle.
- *Poison M + Poison DD*: During a retraining cycle (triggered either by an evasion of DD or a genuine drift signal), the attacker uses poisoned instances to corrupt M . Concurrently, these instances must circumvent DD ’s robustness measures, manipulating its internal representation so it accepts the adversarial distribution as normal. The state of DD must then be maintained using the methods laid out in [Section 4.3](#).
- *Backdoor M + Poison DD*: Following the same methodology as the attack above, the attacker poisons DD (and maintains it) to secure the system’s state. But they implant a specific latent trigger in M during retraining rather than just degrading its general accuracy.
- *Poison DD + Evade M*: To execute a high volume of evasion attacks against M , the attacker must prevent these adversarial inputs from triggering a drift signal. The attacker first poisons DD , raising its tolerance to change. With the detector effectively blinded, the attacker can feed crafted, highly perturbed evasion inputs to M without fear of triggering a defensive retraining cycle.

Write a subsection here (include attacks that require no DD manipulation - ride real drift signal)

Break into more subcategories

- *Backdoor DD + Poison M*: The attacker first establishes a latent trigger in *DD*. When the attacker is ready to poison *M*, they activate the *DD* backdoor to fire a drift signal exactly when their poisoned payload enters the window, effectively guaranteeing *M* retrain on the malicious data.

4.4.2 Pointless or Impractical Combinations

These combinations represent conflicting operational objectives or technical infeasibility.

- *Evade M + Evade DD*: If the attacker’s goal is to cause a misclassification on *M* during inference, evading *DD* adds no value. (The exception is where a mass evasion campaign on *M* is the goal).
- *Backdoor DD + Evade M*: Establishing a backdoor in a complex generative model like the RRBm-DD requires a considerable amount of effort. Using that backdoor merely to facilitate instance-level evasion on *M* (a binary Decision Tree) is a waste of resources.
- *Backdoor DD + Backdoor M*: This requires the adversary to simultaneously train two entirely distinct, hidden triggers into two radically different architectures using the same data stream. Optimising for both simultaneously, without them interfering with one another, is likely practically impossible.

Chapter 5

Implementation

Chapter 6

Evaluation

Chapter 7

Conclusion

Chapter 8

Ethical Issues

Rewrite
ethics
chapter

This chapter considers the wider ethical, legal, and professional implications of the work. The primary ethical concerns posed by this project lie in the fact that the same feedback-guided attack strategies proposed to stress-test and improve system robustness could, in practice, be repurposed by malicious actors to evade detection in currently deployed systems.

8.1 The Dual-Use Dilemma and Offensive Security

The central contribution of this work is a closed-loop attack agent designed to degrade the performance of adaptive learning systems while evading state-of-the-art drift detectors [5, 4]. By definition, this research falls under the umbrella of *offensive security* or *red teaming*.

This comes with an inherent risk that identifying the precise boundaries of current defences could provide a blueprint for attackers. However, it is widely accepted in the security community that "security through obscurity" (relying on the ignorance of attackers) is a fragile defence strategy. As noted in [Chapter 1](#), adaptive systems are increasingly deployed in critical domains such as network intrusion detection and fraud prevention. If these systems harbour latent vulnerabilities to feedback-driven attacks, it is ethically preferable for these weaknesses to be identified and documented by this work rather than be discovered by potential real-world adversaries.

To mitigate the risks associated with this dual-use technology, this project adheres to the principle of defensive disclosure. The attack algorithms are also implemented solely within a sandboxed simulation environment using public datasets, ensuring no live systems are targeted.

8.2 Legal and Professional Compliance

From a legal perspective, this project is conducted in strict accordance with the *Computer Misuse Act 1990 (UK)*. Section 3 of the Act prohibits unauthorised acts with the intent to impair the operation of a computer.

- **Authorisation :** All experiments are conducted on local, isolated hardware or authorised university compute clusters.
- **Target Scope:** The victim systems are local instantiations of open-source algorithms, not external services. At no point does this project interact with, probe, or attempt to subvert third-party servers or commercial APIs.

8.3 Data Governance and Privacy

Finally, the project relies exclusively on publicly available benchmark datasets. No personally identifiable information (PII) is collected, processed, or generated. The risk of privacy violation is therefore negligible. The focus remains strictly on the algorithmic dynamics of drift detection rather than the content of the data itself.

Appendix A

First Appendix

Bibliography

- [1] Gao J, Fan W, Han J, Yu PS. A General Framework for Mining Concept-Drifting Data Streams with Skewed Distributions. In: Proceedings of the 2007 SIAM International Conference on Data Mining. SIAM; 2007. p. 3-14. Available from: <https://doi.org/10.1137/1.9781611972771.1>.
- [2] Gama J, Žliobaitė I, Bifet A, Pechenizkiy M, Bouchachia A. A survey on concept drift adaptation. ACM Computing Surveys. 2014 Mar;46(4):1–37. Available from: <https://doi.org/10.1145/2523813>.
- [3] Barros RSM, Santos SGTC. A large-scale comparison of concept drift detectors. Information Sciences. 2018 Jul;451–452:348–370. Available from: <https://doi.org/10.1016/j.ins.2018.04.014>.
- [4] Kuppa A, Le-Khac NA. Learn to adapt: Robust drift detection in security domain. Computers and Electrical Engineering. 2022 Sep;102:108239. Available from: <https://doi.org/10.1016/j.compeleceng.2022.108239>.
- [5] Korycki Ł, Krawczyk B. Adversarial concept drift detection under poisoning attacks for robust data stream mining. Machine Learning. 2022 Jun;112(10):4013–4048. Available from: <https://doi.org/10.1007/s10994-022-06177-w>.
- [6] Lu J, Liu A, Dong F, Gu F, Gama J, Zhang G. Learning under Concept Drift: A Review. IEEE Transactions on Knowledge and Data Engineering. 2018 Oct;31(12):2346-63. Available from: <https://doi.org/10.1109/TKDE.2018.2876857>.
- [7] Guzy F, Woźniak M. Employing dropout regularization to classify recurring drifted data streams. In: 2020 International Joint Conference on Neural Networks (IJCNN). IEEE; 2020. p. 1–7. Available from: <https://doi.org/10.1109/IJCNN48605.2020.9207266>.
- [8] Gama J, Medas P, Castillo G, Rodrigues P. In: Learning with Drift Detection. Springer Berlin Heidelberg; 2004. p. 286–295. Available from: https://doi.org/10.1007/978-3-540-28645-5_29.
- [9] Bifet A, Gavaldà R. Learning from Time-Changing Data with Adaptive Windowing. In: Proceedings of the 2007 SIAM International Conference on Data Mining. SIAM; 2007. p. 443–448. Available from: <https://doi.org/10.1137/1.9781611972771.42>.
- [10] Song X, Wu M, Jermaine C, Ranka S. Statistical change detection for multi-dimensional data. In: Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD07. ACM; 2007. p. 667–676. Available from: <https://doi.org/10.1145/1281192.1281264>.
- [11] Qahtan AA, Alharbi B, Wang S, Zhang X. A PCA-Based Change Detection Framework for Multidimensional Data Streams: Change Detection in Multidimensional Data Streams. In: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD '15. ACM; 2015. p. 935–944. Available from: <https://doi.org/10.1145/2783258.2783359>.
- [12] Kloft M, Laskov P. Online Anomaly Detection under Adversarial Impact. In: Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics. vol. 9. PMLR; 2010. p. 405-12. Available from: <https://proceedings.mlr.press/v9/kloft10a.html>.

- [13] Sethi TS, Kantardzic M. Handling adversarial concept drift in streaming data. *Expert Systems with Applications*. 2018 May;97:18–40. Available from: <https://doi.org/10.1016/j.eswa.2017.12.022>.